МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №2
по курсу «Операционные системы»

Выполнил: Г. П. Графов
Группа: М8О-203БВ-25
Преподаватель: Е. С. Миронов

Москва, 2026

Цель работы

Приобретение практических навыков создания процессов операционной системы и организации обмена данными между ними посредством неименованных каналов.

Задание

Разработать две отдельные программы: родительский и дочерний процессы. Первая строка, введённая пользователем в консоль родительского процесса, содержит имя выходного файла. Родитель передаёт это имя дочернему процессу при его запуске.

Последующие команды имеют вид «число и перевод строки». Родитель передаёт команды через первый канал, подключённый к стандартному входу дочернего процесса. Дочерний процесс проверяет число на простоту. Составные числа записываются в выходной файл. При получении отрицательного или простого числа оба процесса завершают работу. Проверка числа выполняется на стороне дочернего процесса.

Второй канал используется для передачи статусов от дочернего процесса к родительскому. Необходимо обрабатывать ошибки системных операций и освобождать используемые ресурсы [1].

Принятые уточнения

Реализация выполнена на C++17 с собственным платформенным интерфейсом над API операционных систем. Стандартная библиотека применяется для хранения данных, разбора строк и работы с консолью; создание процессов, каналов и запись выходного файла выполняются средствами ОС.

Команда содержит одно целое число в диапазоне типа `int`. Числа 0 и 1 не являются ни простыми, ни составными: они пропускаются без завершения обработки. Некорректная строка отклоняется, после чего допускается ввод следующей команды. Конец стандартного ввода завершает обмен. Выходной файл создаётся либо обнуляется при запуске дочернего процесса.

Метод решения

Решение разделено на общую логику лабораторной и платформенный слой. Родительская программа организует ввод команд и обмен данными, дочерняя выполняет проверку чисел и запись результата. Вызовы ОС, связанные с процессами, каналами и выходным файлом, вынесены в библиотеку `os`.

Создание процесса и перенаправление потоков

Родитель создаёт два неименованных канала. Первый передаёт строки от родителя к ребёнку, второй возвращает однобайтовые статусы. Стандартный ввод ребёнка связывается с читающим концом первого канала, стандартный вывод — с пишущим концом второго. Диагностические сообщения выводятся в стандартный поток ошибок и не смешиваются со статусами.

Направление	Передаваемые данные	Назначение
Родитель → ребёнок	Строка с числом	Команда
Ребёнок → родитель	Один байт	Статус обработки
Ребёнок → файл	Число и перевод строки	Результат

После запуска ребёнка родитель закрывает свои копии читающего конца первого канала и пишущего конца второго. Родителю остаются только конец отправки команд и конец чтения ответов. Это необходимо, в частности, для корректного обнаружения закрытия канала [4, 5].

Протокол обмена

Дочерний процесс сначала открывает выходной файл и отправляет статус готовности `R`. Только после этого родитель начинает передавать числа. Такая последовательность позволяет обнаружить ошибку открытия файла до начала обработки команд.

Статус	Значение
`R`	Дочерний процесс готов к работе
`W`	Составное число записано в файл
`I`	Число 0 или 1 пропущено
`E`	Ошибка формата числа или выход за диапазон `int`
`S`	Получено простое или отрицательное число

На каждую переданную строку родитель ожидает один статус. Статус немедленно отправляется ребёнком со сбросом буфера вывода. При `S` родитель прекращает ввод, закрывает каналы и ожидает завершения дочернего процесса. Если пользователь завершает ввод, закрытие пишущего конца первого канала позволяет ребёнку обнаружить конец потока.

Проверка простоты

После разбора строки отрицательное значение приводит к завершению работы. Для неотрицательного значения проверяется простота. Значение 2 является простым, остальные чётные числа больше 2 — составными. Для нечётного числа проверяются нечётные делители начиная с 3.

Проверка продолжается, пока выполняется условие $d \leq n/d$. Оно эквивалентно ограничению $d^2 \leq n$ для положительных чисел, но не требует умножения, которое могло бы вызвать переполнение. При обнаружении делителя число признаётся составным. Временная сложность проверки составляет $O(\sqrt{n})$, дополнительная память — $O(1)$.

Описание программы

Структура файлов

Пути приведены относительно корня проекта `Operation-Systems`.

- `lab2/code/parent/main.cpp` — ввод имени файла и команд, запуск ребёнка, приём статусов и ожидание завершения.
- `lab2/code/child/main.cpp` — обработка команд, проверка чисел, отправка статусов и запись результатов.
- `lab2/code/common.cpp` — общий разбор целого числа и проверка простоты.
- `lab2/code/include/common.h` — объявления функций общей логики.

- lab2/code/include/protocol.h — константы статусов обмена.
- lab2/code/include/resources.h — управление временем жизни файловых ресурсов.
- common/code/os/os.h — платформенно-независимый интерфейс.
- common/code/os/os.cpp — полная запись данных и формирование командной строки.
- common/code/os/os_win.cpp — реализация интерфейса для Windows.
- common/code/os/os_posix.cpp — реализация интерфейса для Linux.

Основные типы и функции

os::FileHandle и os::ProcessHandle скрывают платформенное представление ресурсов. В реализации Windows используются дескрипторы HANDLE, в Linux — файловые дескрипторы и идентификаторы процессов. Прикладная логика обращается к ним через функции общего интерфейса.

os::ProcessStartInfo содержит путь программы, аргументы и параметры перенаправления стандартных потоков. os::CreateProc запускает ребёнка и возвращает дескриптор процесса. os::WaitProc ожидает завершения, а os::CloseProc освобождает ресурс процесса.

os::CreatePipe создаёт канал. os::PipeRead читает данные, os::PipeWrite выполняет отдельную запись, os::WriteAll повторяет запись до передачи всех байтов. os::OpenWrite открывает выходной файл на запись. os::CloseFile освобождает файловый ресурс.

Обёртка UniqueFile реализует RAII: при выходе из области видимости вызывает закрытие ресурса. Копирование владельца запрещено, поэтому один ресурс не закрывается двумя независимыми владельцами.

Используемые средства Windows

- CreatePipe, SetHandleInformation — создание каналов и управление наследованием.
- DuplicateHandle — подготовка наследуемых стандартных дескрипторов.
- InitializeProcThreadAttributeList, UpdateProcThreadAttribute, DeleteProcThreadAttri — явный список наследуемых дескрипторов.
- CreateProcessW — запуск дочерней программы с перенаправлением стандартных потоков.
- ReadFile, WriteFile, CreateFileW — ввод-вывод и создание выходного файла.
- WaitForSingleObject, GetExitCodeProcess, CloseHandle — ожидание, проверка кода завершения и освобождение ресурсов.
- GetModuleFileNameW, MultiByteToWideChar, WideCharToMultiByte — определение пути программы и преобразование кодировки путей.

В CreateProcessW передаётся изменяемый буфер командной строки. Путь к ребёнку определяется относительно исполняемого файла родителя. В текущей сборке используются имена lab2_parent.exe и lab2_child.exe. Дескриптор первичного потока закрывается сразу после создания процесса; это не останавливает сам поток. Дескриптор процесса сохраняется для ожидания завершения [3].

Используемые средства Linux

- pipe2, fcntl — создание каналов и настройка FD_CLOEXEC.
- fork, dup2, execv — создание процесса, перенаправление стандартных потоков и запуск программы ребёнка.
- open, read, write, close — работа с файлами и каналами.
- waitpid — получение результата завершения ребёнка.
- readlink — получение пути текущей программы через /proc/self/exe.
- sigaction — настройка обработки SIGPIPE.

Внутри запуска процесса дополнительно используется временный служебный канал для передачи ошибки подготовки или execv родителю. При успешном execv он закрывается благодаря FD_CLOEXEC. Этот канал не участвует в обмене пользовательскими числами.

Обработка ошибок

Ошибки системных операций преобразуются в исключения с названием операции и кодом ОС. Внешние обработчики выводят диагностику в stderr и устанавливают ненулевой код завершения. Родитель проверяет не только статусы протокола, но и код завершения ребёнка.

Частичные записи обрабатываются в WriteAll. В Linux чтение и запись повторяются при EINTR; сигнал SIGPIPE игнорируется, чтобы закрытие читающего конца можно было обработать как ошибку записи, а не аварийное завершение процесса [5].

Сборка и проверка работы

Среда сборки

Сборка выполнена в Windows из PowerShell. В журнале CMake определены генератор Visual Studio 18 2026, компилятор MSVC 19.51.36260.0 и Windows SDK 10.0.26100.0. Использована конфигурация Debug.

Из корня внешнего репозитория OS_Labs выполнены команды:

```
cmake -S .\Operation-Systems -B .\Operation-Systems\build
cmake --build .\Operation-Systems\build --config Debug
```

Сборка завершилась без ошибок компиляции и компоновки. Созданы библиотеки os.lib, lab2_logic.lib и исполняемые файлы lab2_parent.exe, lab2_child.exe.

Запуск программы

Родительская программа запускается командой:

```
.\Operation-Systems\build\bin\lab2\lab2_parent.exe
```

Обе программы должны находиться в одной папке. Имя дочернего исполняемого файла в функции определения пути должно совпадать с именем результата сборки. На этапе проверки обнаружено несовпадение имён `child.exe` и `lab2_child.exe`: оно приводило к ошибке `CreateProcessW`. После согласования имени запуск был подтверждён.

Контрольный пример

Для проверки основного сценария используется следующая последовательность ввода:

```
result.txt
4
6
9
17
```

Ожидаемое содержимое файла:

```
4
6
9
```

Значения 4, 6 и 9 составные и подлежат записи. Значение 17 простое: оно не записывается и завершает обработку. При запуске из корня `OS_Labs` относительный путь `result.txt` относится к этой рабочей папке, а не к папке исполняемого файла.

Дополнительные контрольные случаи

В таблице приведены ожидаемые результаты для повторной проверки. Она описывает набор проверок, а не журнал их выполнения на пользовательском компьютере.

Входные данные	Ожидаемый результат
`8, -4`	Запись 8, завершение на отрицательном значении
`2`	Завершение без записи числа
`0, 1, 4, -1`	Пропуск 0 и 1, запись 4, завершение на -1
`abc, 4, -1`	Сообщение об ошибке ввода, запись 4, завершение
Два числа в строке	Отклонение команды без завершения обмена
Выход за диапазон `int`	Отклонение команды
Конец ввода	Закрытие входного канала и ожидание ребёнка
Недоступный выходной путь	Диагностика ошибки и ненулевой код завершения

В данной работе исследуется корректность межпроцессного обмена. Измерения ускорения и эффективности многопоточного алгоритма не проводились: они не относятся к заданию лабораторной работы №2. Подтверждение сборки Windows не рассматривается как подтверждение запуска Linux-реализации на том же компьютере.

Выводы

В работе реализован обмен между двумя отдельными программами через неименованные каналы. Родитель управляет вводом и последовательностью запросов, а дочерний процесс выполняет проверку простоты и запись составных чисел в файл.

Разделение команд и ответов между двумя каналами позволило согласовать завершение процессов: после статуса остановки родитель больше не передаёт команды. Начальное подтверждение готовности исключает начало обмена до успешного открытия выходного файла.

Корректность работы зависит не только от передачи данных, но и от времени жизни дескрипторов. Закрытие ненужных концов каналов обеспечивает обнаружение конца потока, а ожидание ребёнка позволяет родителю получить окончательный результат его работы. RAII уменьшает число путей выполнения, на которых ресурс мог бы остаться незакрытым.

Платформенный интерфейс отделяет задачу обработки чисел от различий Windows и Linux. При этом переносимость требует согласованных правил владения ресурсами, обработки ошибок и наследования дескрипторов, а не только одинаковых имён функций. Практическая проверка также показала необходимость согласовывать имя дочерней программы в коде с именем, сформированным системой сборки.

Список литературы

[1] Лабораторные работы №2. Методические указания по курсу «Операционные системы». Группа вариантов 1, вариант 5.

[2] Ядров А. Кроссплатформенность. Материалы занятий по курсу «Операционные системы». Репозиторий примера: `Yadroff/Operation-Systems`, GitHub.

[3] Microsoft Learn. CreateProcessW function. Электронный ресурс. `https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-createprocessw`

Дата обращения: 11.11.2026.

[4] Microsoft Learn. Creating a Child Process with Redirected Input and Output. Электронный ресурс. `https://learn.microsoft.com/en-us/windows/win32/procthread/creating-a-child-process-with-redirected-input-and-output`

Дата обращения: 11.11.2026.

[5] Linux man-pages. pipe(7). Электронный ресурс. `https://man7.org/linux/man-pages/man7/pipe.7.html`

Дата обращения: 11.11.2026.